CSC 251 Lecture 02: Assignment, Arithmetic, Output

A bit more on Python Types

Type conversion (casting):

```
int(a)      float(b)      str(5.2)
```

Example 1: (see `Lect02Code.ipynb` for more examples)

```
b = 2.0
print(int(b))
print(b)

c = "3.14"
print(float(c))
print(int(c))

name = "titus"
print(float(name))
```

Memory Allocation

How much memory is allocated for an int? for a float?

You can check with:

```
import sys
sys.getsizeof(3)
```

A Python integer object (PyIntObject) holds the following in memory:

- The value (4 bytes)
- The object size (8)
- The reference count (8) ⇒ 28
- A type pointer (8)

Any worry about integer overflow? (try it!)
https://www.laurentluce.com/posts/python-integer-objects-implementation/

A Python float object (PyFloatObject) follows the IEEE 64-bit double-precision standard:

- The value (8 bytes)
- The reference count (8)
- A type pointer (8) ⇒ 24

https://docs.python.org/3/c-api/float.html

Variable Assignment

Example 2:

What's the output?

```
a = 3
print(a)
a = "hello"
print(a)
a = 3.999999
print(a)
```

Python Arithmetic Operations:

Demo of arithmetic operations (`Lect02Code.ipynb`)

`+  -  *`

float div `/` 3/2 = 1.5

floor div `//` 3//2 = 1 1//2 = 0 -1//2

7 R 1
2 | 15
-14
1

`%` 15 % 2 = 1

`**` 4 ** 2 = 16

Order of Operations:

( )
**
* / // % } L to R
+ -

R to L

Example 3: Evaluate the following:

a) `30-4*5+9/(32%10)` 0.5 4.5

b) `13%12+3**4//2-3*10` 11

c) `2**1**3` 2 8 R to L

Augmented Assignment

As you've seen before (or not):

Operator	Example Usage	Equivalent To
+=	x += 5	x = x + 5
−=	y −= 2	y = y − 2
*=	z *= 10	z = z * 10
/=	a /= b	a = a / b
%=	c %= 3	c = c % 3
//=	x //= 3	x = x // 3
**=	y **= 2	y = y**2

Example 4: Evaluate:

```
x = 10
y = 2
x /= (y + 3)
print(x)
```

2.0

Mixed Type Expressions

When evaluating mathematical expressions:

```
int + int
float * float
int * float
str + int
```

str + str(int)

Constants

Use a named constant (ALL CAPS) for "magic numbers" and values that do not change during execution.

```
TAX_RATE = 0.28
```

Why use constants?

1) Reuse, only code once
2) Easy to read
3) Safe guard

Output

Output statements:

```
print('put your output here')
print("or here")
```

To add an extra line return:

```
print('put your output here\n')
print("or here")
```

To suppress the line return:

```
print('put your output here', end='')
print("or here")
```

To separate arguments with something other than a space:

```
print('yes', 'no', 'maybe', sep='**')
```

Formatting Output with F-strings (Python 3.6 and higher)

F-strings (formatted string literals)

```
fstring = f"Welcome to NCSU"
print(fstring)
Welcome to NCSU
```

Used with replacement fields:

```
uni_name = "NCSU"
print(f"Welcome to {uni_name}")
Welcome to NCSU

month="January"
day=10
print(f"It's {month} {day}, 2024")
It’s January 10, 2024
```

What if you used single quotes around the above print statement?
Python 3.12 and newer allows quote reuse within an f-string

Example 5: What does the output look like? What do we want it to look like? (typesB.py)

```
total_purchase = float(input("Enter total purchase amount: "))
units_ordered = int(input("Enter number of units ordered: "))

price_per_unit = total_purchase/units_ordered
print("Unit price:", price_per_unit)
```

Format to 2 decimal places:

```
print(f"Unit price: {price_per_unit:.2f}")
```

Inserting comma separators:

```
big_num = 4_000_000_000_000
print(f"{big_num:,}")
```

`4,000,000,000,000` versus `4000000000000`

The output specifiers of the f-string following the order:

```
[alignment][width][,][.precision][type]
```

< ^ > # .2 f d e s

For a more complete list of format specifiers:
https://cissandbox.bentley.edu/sandbox/wp-content/uploads/2022-02-10-Documentation-on-f-strings-Updated.pdf

Example 6:

a)

```
num = 95.6789
print(f"The number is {num:>10.3f}")
```

b)

```
lang = "Python"
print(f"{lang:>45}")
```

In a single statement?

Tabular Output

f-strings are extremely useful:

```
print(f"{fahr:>20.2f}|{cels:>20.2f}|{kelvin:>20.2f}")
```

once we get to iteration…

Debugging with F-strings

Value will be output if no ‘:’ but a ‘=’ as in:

```
num = 95.6789
print(f"value of {num =}")
```

Provide an example of how you …

1) Force 3 print statements to output on the same line separated by a comma and a space?

```
print("one", end=", ")
print("two", end=", ")
print("three")
```

≈ end not necessary here

2) Write a single Python print statement across multiple lines of your program? (2 options)

```
print("hello"
      "world",
      "!")
```

```
print("hello"
      "world"
      "!")
```

↖ no commas

```
print("""Hello
world
!""")
```

newlines in output

3) Write a single Python assignment statement across multiple lines of your program? (2 options)

```
x = [
    "a",
    "b"
]
```

```
var = 2 + 10*30 + \
      (10//2)
```

↖ explict line continuation

```
val = (
    1+1)
```

4) Write a single Python print statement to output 2 strings with a space between? (3 options)

```
print("hello", "world")
print("hello" + " " + "world")
print(f"{str1} {str2}")
```

5) Output 3 arguments in a single print statement without spaces between? (2 options)

```
print(f"{x}{y}{z}")
print("one", "two", "three", sep="")
print("one" + "two" + "three")
```

↖ no space

6) Output 3 arguments in a single print statement with a tab between? (3 options)

```
print("python", "is", "great", sep="\t")
print("apple\tbanana\torange")
print(f"{a}\t{b}\t{c}")
```

Reminder: Complete the Experience and Interest Survey by Tuesday at 11:59 PM